

PROGRESSION SYSTEM

OVERVIEW

The progression system oversees the unlocking of new features for the player whenever certain events take place. It's primarily managed by the script `ProgressionManager.cs`, which is sitting on the `ProgressionManager` game object.

HOW THE PROGRESSION SYSTEM WORKS

Besides the `ProgressionManager` class, at the end of the `ProgressionManager.cs` script, there's another short class called "`ProgressEvent`" that holds all the information about a given progress event. The information held by the `ProgressEvent` class includes a boolean function that determines if the progress should occur, an action that defines what the progress event should do, and a float that dictates how long after the event's conditions are met its action should be carried out.

The `ProgressionManager` class holds an array of instances of the `ProgressEvent` class. That array is called "`progressEvents`". Every update, it will check if any progress event's condition has been met. If one of them has been, then the progression manager will start a coroutine that waits until the `ProgressEvent`'s delay time has elapsed to call the event's action. The progression manager also makes sure that it only calls each progress event's action once.

While this is admittedly not the best way the progression system could work, I decided not to rewrite it because the way it works now makes it relatively easy to add new progress events.

ADDING NEW PROGRESS EVENTS

To add a new progress event, you will need to add a new entry **at the end** of the “progressEvents” array in the ProgressionManager. Each entry creates a new instance of the ProgressEvent class. To do that, you use the ProgressEvent class’s constructor. Here’s an example of how the constructor works:

```
new ProgressEvent(() => LevelManager.overMaxCarbon(), () =>
{CarbonDial.current.UnhideCarbonDial();}, 0f)
```

Three parameters are passed in the constructor. The first one is the condition at which the progress event should be called; it has to be a function that returns a bool. The second one is the action that should be called when the condition is met; it has to be a function that returns void. The third parameter is a float that defines the time in seconds between when the condition is met and when the action should be called.

C#’s lambda notation is used to pass the function in the constructor. To define an action/function with it, you do this: “() => ExampleFunction()”. Note that ExampleFunction() can either be an actual function that you’ve defined elsewhere, or you can define a short function right there, like this: “() => exampleVariable > 12”.

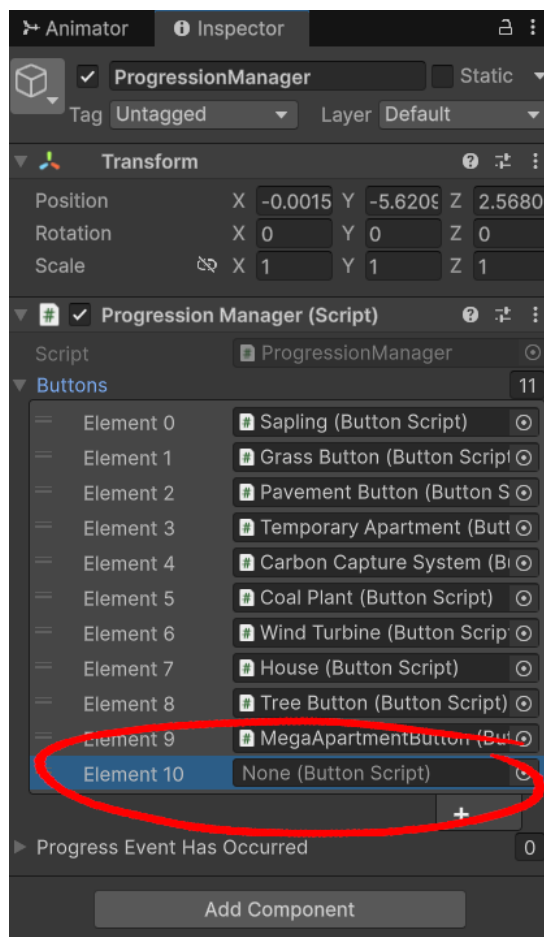
ADDING UNLOCK EVENTS FOR NEW TILES

The process for adding events to unlock new tiles is about the same as adding other progression events. For most of the process, use the previous section “Adding New Progress Events”.

The part that’s slightly different is filling out the action parameter in the new progress event constructor. It should look about like this:

```
() => {TileSelectPanel.TSP.AddButton(buttons[X])}
```

This will unlock the button that corresponds to the button script found at position “X” in the “buttons” array. For this to work, you need to add the button script for the button you want to unlock to the Progression Manager’s “buttons” array in the inspector.



INTERNAL API

The Progression Manager is available at the singleton “ProgressionManager.PM”.

`void CallProgressEvents(int[] progressEventsToCall)`

Calls all progress events that correspond to the integers in the “progressEventsToCall” array. This is primarily only used by the developer cheats system.

-Graydon Bruyn, June 2026